

Module 4 — Splunk: Exploring `eval` Command Functions

Training Notes for Participants

Quick Recap: What is `eval`?

The `eval` command calculates an expression and writes the result into a (new or existing) field. If the field already exists, it gets overwritten.

```
spl
... | eval new_field = expression
```

A few ground rules to share with participants up front:

- Field names and string literals must be wrapped in **double quotes** when they are literal text: `eval city="Bangalore"`.
- Field **values** that are numbers stay unquoted: `eval count=10`.
- You can chain multiple assignments in one `eval` using commas: `... | eval a=1, b=a+2, c=b*3`
- `eval` works **per event (row)** — it does *not* aggregate across events the way `stats` does.

The functions below are grouped exactly as in the module agenda.

1. Conversion Functions

Used to convert values between **strings** and **numbers**, and to format numbers for display.

```
tostring(X, Y)
```

Converts `(X)` to a string. The optional second argument `(Y)` controls formatting:

Format	Effect	Example input	Result
<code>"commas"</code>	Adds thousands separators (2 decimals)	<code>1234567.89</code>	<code>1,234,567.89</code>
<code>"duration"</code>	Converts seconds → <code>HH:MM:SS</code>	<code>615</code>	<code>00:10:15</code>
<code>"hex"</code>	Converts a number → hexadecimal	<code>165</code>	<code>0xa5</code>

```
spl
... | eval bytes_pretty = tostring(bytes, "commas")
... | eval uptime      = tostring(uptime_secs, "duration")
... | eval hex_val     = tostring(165, "hex")
```

`tonumber(NUMSTR, BASE)`

Converts the string `NUMSTR` to a number. `BASE` is optional (default 10). If conversion fails, it returns NULL.

```
spl
... | eval n    = tonumber("123.45")      | -> 123.45
... | eval hx   = tonumber("0A4", 16)     | -> 164
```

Trainer tip: A common use is forcing a string-typed field into numeric form so it can be used in math or sorted correctly: `eval response_ms = tonumber(response_ms)`.

2. Date and Time Functions

Splunk stores event time internally as **epoch** (seconds since 1 Jan 1970) in the `_time` field. These functions convert between epoch and human-readable strings.

`now()` and `time()`

- `now()` — current time as epoch, fixed at the moment the search starts.
- `time()` — current time as epoch **with microsecond precision**, evaluated per event.

`strftime(X, Y)` — epoch → string

Formats epoch value `X` using format string `Y`.

```
spl
... | eval readable = strftime(_time, "%Y-%m-%d %H:%M:%S")
```

`strptime(X, Y)` — string → epoch

Parses string `X` into epoch using format `Y`. (Note the **p** = parse.)

```
spl
```

```
... | eval epoch = strptime("2024-03-15 10:30:00", "%Y-%m-%d %H:%M:%S")
```

`relative_time(X, Y)`

Takes epoch `(X)` and applies a relative-time modifier `(Y)`, returning a new epoch.

```
spl

... | eval yesterday_start = relative_time(now(), "-1d@d")
... | eval start_of_hour    = relative_time(now(), "@h")
```

Common `strftime` / `strptime` format specifiers

Code	Meaning	Example
<code>%Y</code>	4-digit year	2024
<code>%m</code>	Month (01-12)	03
<code>%d</code>	Day of month (01-31)	15
<code>%H</code>	Hour, 24-hour (00-23)	14
<code>%M</code>	Minute (00-59)	30
<code>%S</code>	Second (00-59)	05
<code>%p</code>	AM / PM	PM
<code>%A</code>	Full weekday name	Friday
<code>%B</code>	Full month name	March
<code>%j</code>	Day of year (001-366)	075
<code>%Z</code>	Time zone	IST

3. String (Text) Functions

`len(X)` — length of a string

```
spl
```

```
... | eval name_length = len(username)
```

lower(X) / **upper(X)** — **change case**

```
spl
... | eval host_upper = upper(host)
... | eval user_lower = lower(User)
```

trim(X, Y) / **ltrim(X, Y)** / **rtrim(X, Y)**

Removes characters from both ends / left / right. If **(Y)** is omitted, whitespace is trimmed.

```
spl
... | eval clean = trim("  hello  ")      | -> "hello"
... | eval x      = ltrim("ZZZabc", "Z")   | -> "abc"
```

replace(X, Y, Z)

Replaces matches of **regex** **(Y)** in string **(X)** with **(Z)**. Backreferences **(\1, \2, ...)** are supported.

```
spl
... | eval iso_date = replace(date, "(\d+)/(\d+)/(\d+)", "\3-\1-\2")
| Converts 03/15/2024 -> 2024-03-15
```

substr(X, Y, Z)

Returns a substring. **(Y)** is the **1-based** start position; **(Z)** is optional length. A negative **(Y)** counts from the end.

```
spl
... | eval first3 = substr("splunk", 1, 3)   | -> "spl"
... | eval last3  = substr("splunk", -3)     | -> "unk"
```

split(X, Y)

Splits string **(X)** by delimiter **(Y)** into a **multivalue** field.

```
spl
```

```
... | eval parts = split("a,b,c", ",") | -> [a, b, c]
```

`urldecode(X)`

Decodes a URL-encoded string.

```
spl  
  
... | eval url = urldecode("http%3A%2F%2Fwww.splunk.com")  
| -> http://www.splunk.com
```

4. Comparison and Conditional Functions

This is the most heavily used group in real searches — branching logic, defaults, and matching.

`if(X, Y, Z)`

If condition `(X)` is true, return `(Y)`; otherwise return `(Z)`.

```
spl  
  
... | eval status = if(error=1, "Failed", "OK")
```

`case(X1, Y1, X2, Y2, ...)`

Evaluates conditions in order and returns the value paired with the **first true** condition. Returns NULL if none match.

```
spl  
  
... | eval level = case(  
    status<300, "Success",  
    status<400, "Redirect",  
    status<500, "Client Error",  
    status>=500, "Server Error")
```

`validate(X1, Y1, X2, Y2, ...)`

The logical opposite of `(case)` — returns the value paired with the **first FALSE** condition. Useful for input validation / error messages.

```
spl
```

```
... | eval msg = validate(
    score>=0, "Score is below zero",
    score<=100, "Score is above 100")
```

`coalesce(X, ...)`

Returns the **first non-NULL** argument. Great for normalizing field names from different sources.

```
spl
... | eval ip = coalesce(src_ip, source_ip, clientip)
```

`cidrmatch("CIDR", IP)`

Returns true if `IP` falls within the given CIDR subnet.

```
spl
... | eval zone = if(cidrmatch("192.168.0.0/16", clientip), "internal", "extern
```

`like(TEXT, PATTERN)`

SQL-style matching. `%` matches any sequence of characters; `_` matches a single character.

```
spl
... | where like(email, "%@gmail.com")
```

`match(SUBJECT, REGEX)`

Returns true if `SUBJECT` matches the regular expression.

```
spl
... | eval valid_phone = if(match(phone, "^\\d{10}$"), "valid", "invalid")
```

`searchmatch(X)`

Returns true if the event matches the search string `X`.

```
spl
... | eval is_target = if(searchmatch("error status=500"), "yes", "no")
```

Helpers: `true()`, `false()`, `null()`, `nullif(X, Y)`

- `true()` / `false()` — return boolean literals (useful in `case` / `if`).
 - `null()` — returns NULL.
 - `nullif(X, Y)` — returns NULL if `X == Y`, otherwise returns `X`.
-

5. Informational Functions

These **test the type or presence** of a value and return a boolean (or a type name).

Presence checks

```
spl
... | eval has_value = if(isnotnull(field), "yes", "no")
... | eval missing   = if(isnull(field),    "yes", "no")
```

Type checks

Function	True when X is...
<code>isnum(X)</code>	a number
<code>isint(X)</code>	an integer
<code>isstr(X)</code>	a string
<code>isbool(X)</code>	a boolean

```
spl
... | eval numeric_check = if(isnum(value), "number", "not a number")
```

`typeof(X)` — returns the type as a string

Possible return values: `Number`, `String`, `Boolean`, `Invalid` (for unknown fields).

```
spl
```

```
... | eval t1 = typeof(12)      | -> "Number"
... | eval t2 = typeof("abc")   | -> "String"
... | eval t3 = typeof(1==1)    | -> "Boolean"
... | eval t4 = typeof(badfield) | -> "Invalid"
```

6. Statistical Functions

Important distinction to emphasize: these `eval` versions of `max()`/`min()` compare **fields *within the same event*** (row-wise). They are *not* the same as `stats max()` / `stats min()`, which aggregate **across events**.

`max(X, ...)` and `min(X, ...)`

Return the largest / smallest of the supplied arguments. Numeric values compare numerically; mixed/non-numeric values compare lexicographically.

```
spl

... | eval highest = max(score1, score2, score3)
... | eval lowest  = min(score1, score2, score3)
```

`random()`

Returns a pseudo-random integer between 0 and $2^{31}-1$.

```
spl

... | eval dice = (random() % 6) + 1      | -> 1 through 6
```

7. Mathematical Functions

Function	Purpose	Example	Result
<code>abs(X)</code>	Absolute value	<code>abs(-42)</code>	42
<code>ceiling(X)</code> (alias <code>ceil</code>)	Round up	<code>ceiling(1.2)</code>	2
<code>floor(X)</code>	Round down	<code>floor(1.9)</code>	1
<code>round(X, Y)</code>	Round to Y decimals (default 0)	<code>round(3.14159, 2)</code>	3.14

Function	Purpose	Example	Result
<code>sqrt(X)</code>	Square root	<code>sqrt(9)</code>	3
<code>pow(X, Y)</code>	X raised to power Y	<code>pow(2, 10)</code>	1024
<code>exp(X)</code>	e raised to X	<code>exp(1)</code>	2.718...
<code>ln(X)</code>	Natural log (base e)	<code>ln(2.718)</code>	~1
<code>log(X, Y)</code>	Log of X, base Y (default 10)	<code>log(100)</code>	2
<code>pi()</code>	Constant π	<code>pi()</code>	3.14159...
<code>sigfig(X)</code>	Round to significant figures	<code>sigfig(1.00500*2)</code>	2.01

```
spl
```

```
... | eval gb          = round(bytes/1024/1024/1024, 2)
... | eval magnitude = floor(log(value, 10))
```

Putting It Together — A Realistic Example

A single search that touches several function groups — useful as a live demo:

```
spl
```

```
index=web sourcetype=access_combined
| eval response_kb  = round(bytes/1024, 2)
| eval client_zone  = if(cidrmatch("10.0.0.0/8", clientip), "Internal", "External")
| eval status_class = case(
    status<300, "Success",
    status<400, "Redirect",
    status<500, "Client Error",
    status>=500, "Server Error")
| eval event_time    = strftime(_time, "%Y-%m-%d %H:%M:%S")
| eval host_upper    = upper(host)
| eval user          = coalesce(user, username, "anonymous")
| table event_time, host_upper, client_zone, status, status_class, response_kb,
```

This demonstrates, in order: a **mathematical** calculation, a **comparison/conditional** with `cidrmatch` and `case`, a **date/time** conversion, a **string** transformation, and a **conditional**

default via `coalesce`.

Practice Exercises for Participants

1. **Conversion:** Convert a `duration_secs` field into `HH:MM:SS` and into a comma-formatted number.
 2. **Date/Time:** Create a field showing only the **weekday name** for each event.
 3. **String:** Extract the **domain** portion of an email field using `substr` + `match`, or `split`.
 4. **Conditional:** Use `case` to bucket `response_time` into Fast (<200ms), Normal (<1s), Slow (≥1s).
 5. **Informational:** Flag every event where a critical field `isnull`.
 6. **Statistical:** Given three score fields, output the highest and lowest per event.
 7. **Mathematical:** Convert bytes to GB rounded to 3 decimal places.
-

Quick Reference Cheat Sheet

Category	Key Functions
Conversion	<code>tostring</code> , <code>tonumber</code>
Date & Time	<code>now</code> , <code>time</code> , <code>strftime</code> , <code>strptime</code> , <code>relative_time</code>
String	<code>len</code> , <code>lower</code> , <code>upper</code> , <code>trim</code> , <code>replace</code> , <code>substr</code> , <code>split</code> , <code>urldecode</code>
Comparison & Conditional	<code>if</code> , <code>case</code> , <code>validate</code> , <code>coalesce</code> , <code>cidrmatch</code> , <code>like</code> , <code>match</code> , <code>searchmatch</code> , <code>nullif</code>
Informational	<code>isnull</code> , <code>isnotnull</code> , <code>isnum</code> , <code>isint</code> , <code>isstr</code> , <code>isbool</code> , <code>typeof</code>
Statistical	<code>max</code> , <code>min</code> , <code>random</code>
Mathematical	<code>abs</code> , <code>ceiling</code> , <code>floor</code> , <code>round</code> , <code>sqrt</code> , <code>pow</code> , <code>exp</code> , <code>ln</code> , <code>log</code> , <code>pi</code> , <code>sigfig</code>
